

Limnalis v0.2.2

A Disclosure Language for Complex Systems

Dave Medeiros · Panoptic Systems

March 2026

Contents

Limnalis v0.2.2	3
Reader's Guide: Domain and Purpose	3
Reader's Guide: The Execution Model	4
Abstract Machine Signature	4
Step Context	4
Machine State	4
Uniform Primitive Envelope	4
Primitive Instruction Set	5
0. Architectural Layers	5
0.1 Notation-Layer Responsibilities	5
1. Core Design Rule	5
2. Canonical Kernel	6
2.1 Expression Forms	8
2.2 Reference Resolution	8
3. Claims and Strata	8
3.1 Block Aggregation	8
4. Evaluation Semantics	8
4.1 Field Semantics	9
4.2 Multi-Evaluator Model	9
4.3 Resolution Policies	9
4.4 Evidence Conflict	9
4.5 Reason Taxonomy	9
4.6 Block Aggregation Under Multiple Evaluators	10
4.7 Symbol / Status Split	10
5. Four-Valued Logic	10
6. Frames, Patterns, and Facet Policies	10
6.1 Frame Operations	11
7. Bridges, Boundary Crossings, and Transport Semantics	11
7.1 Transport Modes	12
7.2 Transport Result	12

7.3 Transport Lint	13
8. Baselines, Unbound Behavior, and Emergence	13
8.1 Session-Relative Baseline Timing	13
8.2 Unbound Behavior	14
8.3 Emergence	14
9. Anchors, Adequacy, and Model-Status	14
9.1 Assessment-Level Semantics	15
9.2 Multiple Assessments Per Task	15
9.3 Licensing Rules	15
9.4 License Result	15
9.5 Circularity Rule	16
10. Judgments and Normative Terms	16
11. Surface Syntax and Operator Kernel	16
11.1 Surface Sugar	16
11.2 Minimal Operator Kernel	16
12. Worked Example	17
13. Lint Rules	19
13.1 Normative Diagnostic Codes	20
14. Open Extensions	20
15. How to Read Limnalis	20
Appendix A: Grammar and Canonical AST	20
A.1 Parsing Model	20
A.2 Lexical Profile	21
A.3 Top-Level Structure	21
A.4 Frames	21
A.5 Evaluators, Policies, Time, Bindings, and Facet Policies	22
A.6 Assumptions, Baselines, and Evidence	22
A.7 Anchors, Adequacy, and Bridges	23
A.8 Claim Blocks and Claims	24
A.9 Expression Grammar	24
A.10 Canonical AST Nodes	25
A.11 Normalization Rules	28
A.12 Resolution Rules	28
16. Reference Evaluator	28
16.1 Purpose	28
16.2 Inputs and Outputs	28
16.3 Effective Step Context	29
16.4 Runtime Artifacts	29
16.5 Bound Handler Contracts	30
16.6 Normative Phase Order	31
16.7 Phase Details	31
Phase 1 — Reference Resolution	31
Phase 2 — Step Context	31
Phase 3 — Baseline Service	31
Phase 4 — Adequacy and Licensing	31
Phase 5 — Evidence View	31

Phase 6 — Claim Truth Per Evaluator	31
Phase 7 — Support Per Evaluator	32
Phase 8 — Aggregate Claims	32
Phase 9 — Fold Blocks	32
Phase 10 — Transport	32
16.8 Failure Modes	32
16.9 Conformance Rules	32
17. Conformance Corpus	33
17.1 Fixture Conventions	33
17.2 Track A — Core Semantics	33
17.3 Track B — Domain Bundles	34
18. Schema Package	34

Recovery notice. This file is a high-fidelity reconstruction of the lost Limnalis v0.2.2 specification. It was rebuilt from the surviving v0.2 and v0.2.1 PDFs, the v0.2.2 conformance matrix, fixture corpus, AST and result schemas, rendered pages from the missing v0.2.2 PDF, and the specification text and amendments preserved in the project conversation. The semantic content is substantially recoverable; exact original wording, pagination, and binary identity cannot be guaranteed.

Limnalis v0.2.2

Consolidated specification — reconstructed canonical draft

Limnalis is a disclosure language: a formal system whose primary job is to make hidden assumptions visible and checkable. Every statement carries its scope, its modeling anchors, its adequacy conditions, its evaluator and evidence context, and its boundary crossings. It does not merely say, “this is true.” It says, “this is true here, under these assumptions, according to this evaluator and evidence, and here is how you would know when it stops being true.”

Most specification failures are not failures of assertion. The invariant may have been correctly stated. They are failures of undisclosed assumption: the invariant was true where it was written and silently false where it was applied. Limnalis makes context a first-class citizen of every claim.

Major features in v0.2.2: four-layer architecture; seven-part separation; Belnap-Dunn four-valued semantics; frame algebra; fictional-anchor licensing; multi-evaluator panels; resolution policies; transport truth modes; session-based evaluation; assessment-based adequacy; typed diagnostics; canonical EBNF and AST; normalization rules; reference evaluator; conformance corpus; and machine-checkable schemas.

Reader’s Guide: Domain and Purpose

Limnalis is not an AI-governance-only framework. It is a domain-agnostic disclosure language for complex systems, especially where claims depend on frame, assumptions, evidence, evaluators, idealizations, and abstraction boundaries.

It sits at the intersection of:

- formal specification,
- systems modeling,
- governance and assurance,
- evidence-aware evaluation,
- and cross-model or cross-frame reasoning.

It is best understood as three things at once:

1. a **type system for framed claims**,
2. a **contract language for models**, and
3. a **systems DSL for epistemic hygiene**.

Limnalis does not replace mathematics, physics, probability, simulation, or code. It wraps them in an explicit account of where a claim holds, what assumptions are active, what fictions are being used, who or what is evaluating, what evidence exists, and how failure at the edge is represented rather than hidden.

Reader's Guide: The Execution Model

The execution model is a typed evaluator micro-op machine rather than classical three-address code. Each primitive runs against an explicit step context, machine state, and services, and produces typed outputs plus diagnostics.

Abstract Machine Signature

At claim level:

```
[[claim]] : StepContext × MachineState × Services × History
          → ClaimResult × MachineState
```

At expression level, internal and delegated expression evaluation produces `TruthCore`. Claim evaluation is not merely `Context × History → Eval` because sessions and steps affect baseline timing, diagnostics are observable, licensing is claim-local, non-evaluable claims exist, and every evaluable claim yields per-evaluator results before aggregation.

Step Context

```
StepContext:
  frame: FrameNode
  evaluators: [EvaluatorNode]
  resolutionPolicy: ResolutionPolicyNode
  time: TimeCtxNode?
  history: OpaqueHistory
  activeFacetPolicy: FrameFacetPolicyNode?
  sessionId: SessionId
  stepId: StepId
```

The effective context is computed from the bundle, session, step, and environment. Later frame facets override earlier ones. Step time has precedence over session base time, bundle time, and environment clock. A step-specific history binding has precedence over environment history.

Machine State

```
MachineState:
  resolutionStore: ResolutionStore
  baselineStore: Map<BaselineKey, BaselineState>
  adequacyStore: Map<AdequacyKey, AdequacyResult>
  evidenceViews: Map<ClaimId, ClaimEvidenceView>
  diagnostics: [Diagnostic]
```

Machine state is part of evaluator semantics, not merely an optimization detail.

Uniform Primitive Envelope

```
op(inputs, step_ctx, machine_state, services)
  → (output, machine_state, diagnostics)
```

Primitive Instruction Set

1. **resolve_ref** — resolve local and external references. *Internal semantics with a pluggable resolver backend.*
2. **build_step_context** — build effective frame, time, history, evaluator panel, and policy. *Internal.*
3. **resolve_baseline** — resolve a fixed, on-reference, or tracked baseline. *Hybrid.*
4. **evaluate_adequacy_set** — execute and aggregate adequacy assessments for one task. *Hybrid.*
5. **compose_license** — compose individual and exact-set joint adequacy into a license. *Internal.*
6. **build_evidence_view** — materialize declared claim-local evidence and relations. *Internal.*
7. **classify_claim** — decide whether a claim is evaluable. *Internal.*
8. **eval_expr** — evaluate an expression for one evaluator. *Hybrid dispatcher.*
9. **synthesize_support** — determine support and confidence. *Hybrid.*
10. **assemble_eval** — form a complete per-evaluator `EvalNode`. *Internal.*
11. **apply_resolution_policy** — aggregate per-evaluator evaluations. *Hybrid.*
12. **fold_block** — fold per evaluator, then aggregate the block result. *Internal.*
13. **execute_transport** — execute one step-scoped bridge transport query. *Hybrid.*

The fully internal semantic operations are `build_step_context`, `compose_license`, `build_evidence_view`, `classify_claim`, `assemble_eval`, and `fold_block`. Hybrid operations retain internal orchestration while delegating domain-specific leaves.

0. Architectural Layers

Limnalis has four architectural layers.

World layer. Claims about systems, entities, mechanisms, trajectories, thresholds, and emergent behavior. Expressed mainly through `local` and `systemic` claims.

Knowledge layer. Evaluation: who or what is evaluating, from what evidence, with what support, confidence, and provenance. Expressed through `Evaluator`, `Evidence`, `EvidenceRelation`, `Eval`, and epistemic frame facets such as `observer`.

Fiction layer. Assumptions, idealizations, placeholders, proxies, aggregates, and adequacy judgments. Expressed through `Assumption`, `Anchor`, `JointAdequacy`, and license checks.

Notation layer. Authored syntax, symbols, ASCII aliases, normalization, serialization, and stable bindings to external artifacts.

These layers are not the same as the `local` / `systemic` / `meta` strata. Strata organize claims by descriptive level; layers organize the architecture of the language.

0.1 Notation-Layer Responsibilities

The notation layer is representational rather than truth-bearing. It has three responsibilities:

1. **Representation** — authored source forms, operator glyphs, ASCII aliases, shorthand, and binding references.
2. **Canonicalization** — normalization from authored forms to canonical AST, including alias normalization, shorthand expansion, and stable block identity.
3. **Interchange** — machine-readable schemas and stable external-artifact references for tooling, storage, and validation.

The notation layer does not determine world truth, evidence support, or fiction licensing. It makes those layers authorable, canonicalizable, and interoperable.

1. Core Design Rule

Every Limnalis statement separates seven things that are otherwise prone to collapse into one sentence:

1. **Proposition** — what is being claimed.

2. **Frame** — where the claim is supposed to hold.
3. **Assumptions** — what is being taken as active.
4. **Model-status** — which terms are literal, idealized, proxy-like, aggregate, or placeholder.
5. **Evaluator** — what process, agent, institution, model, or policy assigns truth and support.
6. **Evidence** — what the evaluator has to work with.
7. **Evaluation** — the resulting T / F / B / N, plus why.

2. Canonical Kernel

The kernel is a typed bundle, not just a line of notation.

Bundle:

```

id: BundleId
frame: Frame
evaluators: [Evaluator]
resolution_policy: ResolutionPolicy
time: TimeCtx?
claims: [Claim]
assumptions: [Assumption] = []
baselines: [Baseline] = []
anchors: [Anchor] = []
joint_adequacies: [JointAdequacy] = []
bridges: [Bridge] = []
bindings: [Binding] = []
evidence: [Evidence] = []
evidence_relations: [EvidenceRelation] = []
facet_policies: [FrameFacetPolicy] = []

```

A legacy single evaluator declaration normalizes to a singleton evaluator panel plus a single resolution policy.

Frame:

```

system: Symbol
namespace: Symbol
scale: Symbol
task: Symbol
regime: Symbol
observer: Symbol?
version: Symbol?
facet_policy: FrameFacetPolicyRef?

```

Evaluator:

```

id: EvaluatorId
kind: model | human | agent | institution | ensemble | process
role: primary | adversarial | audit | auxiliary?
binding: BindingRef
evidence_policy: BindingRef?
inference_policy: BindingRef?
provenance_policy: BindingRef?

```

ResolutionPolicy:

```

id: ResolutionPolicyId
kind: single | paraconsistent_union | priority_order | adjudicated
members: [EvaluatorId]?
order: [EvaluatorId]?
binding: BindingRef?

```

```

TimeCtx:
  kind: point | interval | window
  t: Timestamp?
  start: Timestamp?
  end: Timestamp?
  lag: Duration?
  step: Duration?

Claim:
  id: ClaimId
  stratum: local | systemic | meta
  kind: atomic | causal | dynamic | emergence | declaration | judgment | note | logi
  expr: Expr
  uses_anchors: [AnchorId] = []
  semantic_requirements: [Property] = []
  annotations: Map<Symbol, Value> = {}
  refs: [BindingRef | EvidenceRef] = []
  eval: Eval?

Assumption:
  id: AssumptionId
  expr: Expr
  status: active | suspended | counterfactual
  refs: [BindingRef] = []

Evidence:
  id: EvidenceId
  kind: measurement | dataset | testimony | simulation | audit | derived
  binding: BindingRef
  observer: Symbol?
  time: TimeCtx?
  completeness: [0,1]?
  internal_conflict: [0,1]?

EvidenceRelation:
  id: EvidenceRelationId
  lhs: EvidenceRef
  rhs: EvidenceRef
  kind: corroborates | conflicts | depends_on | duplicate_of
  score: [0,1]?
  refs: [BindingRef] = []

Binding:
  id: BindingId
  kind: equation | dataset | code | model | document | policy | ontology
  target: URI
  version: Symbol?
  hash: String?

```

`semantic_requirements` participates only in bridge truth modes; it does not affect ordinary claim truth. `license_task` is a reserved annotation key.

2.1 Expression Forms

```
Expr :=
  Predicate(name, args...)
| Logical(op, args...) # not | and | or | implies | iff
| Causal(lhs, rhs, mode=obs|do, intervention?)
| Dynamic(subject, op, target, qualifiers?)
| Emergence(property, onset, persists_while?, dissolves_when?, hysteresis?)
| Declaration(term, declared_as, within?)
| Judged(expr, criterion_ref)
| Note(text)
```

2.2 Reference Resolution

Limnalis permits bundle-local and external references.

```
resolve(ref, bundle, env) → object | failure
```

Required unresolved references emit diagnostics and localize dependent evaluation to `N[missing_binding]` or `N[missing_policy]`. Bridge endpoint patterns are not required to resolve to full frames during static validation.

3. Claims and Strata

`local`, `systemic`, and `meta` are claim strata, not separate semantic universes.

- **local** — entities, components, immediate mechanisms and state changes.
- **systemic** — aggregates, distributions, attractors, thresholds, phase behavior, and emergence.
- **meta** — claims about claims, frames, evaluators, evidence policy, assumptions, anchors, bridges, baselines, and notes.

Only typed meta records are semantically active. `note("...")` is inert and cannot silently alter truth conditions.

Claims inherit bundle frame and time unless overridden by an explicit step context or transport context.

3.1 Block Aggregation

A block's default truth is the fold of four-valued conjunction over its evaluable claims.

```
fold(∧, evaluator-local claim truths)
```

Under multiple evaluators, blocks are folded per evaluator first, then those evaluator-local block truths are aggregated under the bundle resolution policy. `NoteExpr` claims are excluded. An empty evaluable set yields `N[empty_block]`.

4. Evaluation Semantics

At claim level:

```
[[claim]] : StepContext × MachineState × Services × History
          → ClaimResult × MachineState
```

`Eval`:

```
truth: T | F | B | N
reason: Reason?
support: supported | partial | conflicted | absent | inapplicable
confidence: [0,1]?
provenance: [BindingRef | EvidenceRef | EvaluatorId]
```


4.1 Field Semantics

- **truth** — evaluator-relative and frame-relative; not an unqualified metaphysical label.
- **support** — the evidence situation under the active evidence policy.
- **confidence** — a scalar supplied by an evaluator or policy; not truth.
- **provenance** — traceability to evaluator, methods, policies, and evidence.

4.2 Multi-Evaluator Model

Every evaluable claim is computed per evaluator before cross-evaluator aggregation.

```
ClaimResult:
  claimId: string
  evaluable: true | false
  per_evaluator: { EvaluatorId: EvalNode }?
  aggregate: EvalNode?
  license: LicenseResult?
  evidenceView: string?
  diagnostics: [Diagnostic]
```

`Frame.observer` and `evaluator` are distinct. They may coincide but never by silent assumption.

4.3 Resolution Policies

single. Exactly one member; copy its complete `EvalNode`.

paraconsistent_union. Treat truth values as truth/falsity pairs and aggregate by componentwise OR. $T + F$ produces `B[evaluator_conflict]`; $T + N$ produces `T`; all `N` produces `N`. Support becomes conflicted if any evaluator is conflicted or if the aggregate is `B[evaluator_conflict]`; otherwise partial if any is partial; otherwise supported if any is supported; otherwise inapplicable if all are inapplicable; otherwise absent. Confidence is unset by default. Provenance is the deterministic union of participating provenance.

priority_order. Use the first evaluator in declared order whose truth is not `N`; copy its complete evaluation. If all are `N`, aggregate `N`.

adjudicated. A bound adjudicator receives the complete per-evaluator evaluation map and returns an aggregate `EvalNode`.

4.4 Evidence Conflict

Internal conflict belongs on `Evidence.internal_conflict`. Cross-evidence conflict belongs in `EvidenceRelation(kind=conflicts)`. Any evaluator-inferred conflict must be policy-authorized and represented in provenance.

4.5 Reason Taxonomy

Mandatory `B` reasons include:

```
source_conflict, model_conflict, boundary_mix, aggregation_reversal,
observer_split, temporal_smear, self_reference, logical_composition,
evaluator_conflict, adequacy_conflict, method_conflict
```

Mandatory `N` reasons include:

```
out_of_scope, undefined_term, type_error, missing_binding, missing_policy,
missing_evidence, missing_joint_adequacy, uninstantiated, transport_missing,
transport_loss, transport_precondition, transport_mapping_missing,
not_yet_applicable, unsafe_projection, empty_block, logical_composition,
circular_dependency
```

Optional F annotations include:

`refuted`, `threshold_not_met`, `joint_inadequacy`

B and N always require reason codes because those outcomes remain underdetermined without them. F does not require a reason by default; an F reason is attached when the mode of falsity is operationally important.

4.6 Block Aggregation Under Multiple Evaluators

The evaluator does not aggregate claims first and then fold. It folds each evaluator’s claim truths first, then aggregates the evaluator-local block truths. This prevents cross-evaluator conjunction artifacts that no evaluator endorsed.

```
BlockResult:
  blockId: string
  stratum: local | systemic | meta
  per_evaluator: { EvaluatorId: T | F | B | N }
  aggregate: T | F | B | N
  claimIds: [string]
```

4.7 Symbol / Status Split

$\perp$ is a surface token for a paradox-marked expression or state. $\emptyset$ is a surface token for undefinedness. B and N are evaluation outcomes, not the same objects as those symbols.

5. Four-Valued Logic

Limnalis uses Belnap-Dunn style truth/falsity pairs:

T = (1, 0)
 F = (0, 1)
 B = (1, 1)
 N = (0, 0)

For $X = (tX, fX)$ and $Y = (tY, fY)$:

$\neg X = (fX, tX)$
 $X \wedge Y = (tX \wedge tY, fX \vee fY)$
 $X \vee Y = (tX \vee tY, fX \wedge fY)$
 $X \rightarrow Y = \neg X \vee Y$
 $X \leftrightarrow Y = (X \rightarrow Y) \wedge (Y \rightarrow X)$

$\wedge$	T	F	B	N
T	T	F	B	N
F	F	F	F	F
B	B	F	B	F
N	N	F	F	N

The counterintuitive case $B \wedge N = F$ is intentional. B contributes truth and falsity; N contributes neither. Conjunction loses truth-support because one side contributes none, while falsity-support remains because B contributes it. This is a property of the strict connective, not a natural-language claim that “paradox and undefined” are ordinarily false.

6. Frames, Patterns, and Facet Policies

A full `Frame` is a resolved evaluation context. A `FramePattern` is a partial facet assignment used for shorthand, projection, declarations, baseline-local context, and bridge endpoints.

```

FramePattern:
  facets: Map<Facet, Symbol>
  facet_policy: FrameFacetPolicyRef?

```

```
Facet := system | namespace | scale | task | regime | observer | version
```

Canonical frame form:

```

@{system=PowerGrid, namespace=ACLoadFlow, scale=micro,
  task=operations, regime=contingency, version=v2}

```

Legacy shorthand:

```
@PowerGrid:ACLoadFlow::contingency
```

The shorthand normalizes to a partial `FramePattern` with `system`, `namespace`, and `regime`; it is not a full frame.

```

FrameFacetPolicy:
  id: FrameFacetPolicyId
  order:
    system: eq | PartialOrderRef
    namespace: eq | PartialOrderRef
    scale: eq | PartialOrderRef
    task: eq | PartialOrderRef
    regime: eq | PartialOrderRef
    observer: eq | PartialOrderRef
    version: eq | PartialOrderRef
  independent: Set<(Facet, Facet)> = {}
  depends_on: Set<(Facet, Facet)> = {}

```

The conservative default is equality ordering with no independence assumed.

6.1 Frame Operations

```

compatible(f1, f2)
refines(f1, f2)      or f1  $\sqsubseteq$  f2
join(f1, f2)
project(f, S)         $\rightarrow$  FramePattern
resolve(pattern, env)  $\rightarrow$  Frame
matches(frame, pattern, policy)

```

- `compatible` means no facet assignments conflict and no declared dependency is violated.
- `f1  $\sqsubseteq$  f2` means each facet in `f1` equals or refines the corresponding facet in `f2` under the active policy.
- `join` exists only for compatible frames/patterns.
- `project` is a descriptive act: it returns a partial pattern.
- `resolve` completes a pattern into a full frame or fails.

Projection does not silently transport truth. Reusing truth after projection requires declared facet independence or an explicit bridge; otherwise the result is `N[unsafe_projection]`.

Truth does not automatically move upward, downward, or sideways across frames. It moves only by exact frame match or an explicit bridge with sufficient transport semantics.

7. Bridges, Boundary Crossings, and Transport Semantics

A bridge is a rule for relating claims between frames. It is not the same as a trajectory event.

```

Bridge:
  id: BridgeId
  from: FramePattern
  to: FramePattern
  via: BindingRef
  preserve: [Property]
  lose: [Property]
  gain: [Property] = []
  risk: [aggregation_reversal | aliasing | temporal_smear | observer_shift] = []
  transport:
    mode: metadata_only | preserve | degrade | remap_recompute
    claim_map: BindingRef?
    truth_policy: BindingRef?
    preconditions: [BindingRef | Expr] = []
    dst_evaluators: [EvaluatorId]?
    dst_resolution_policy: ResolutionPolicyRef?

```

A legacy bridge without an explicit transport block defaults to `metadata_only`.

`~` is a crossing event relative to a boundary. Bridge is a transport rule. `}` marks a hard boundary beyond which the current frame no longer licenses evaluation; without an activated destination frame or bridge, crossing yields `N[out_of_scope]`.

7.1 Transport Modes

metadata_only. No truth transfer. The result carries the destination pattern, preserve/lose/gain/risk declarations, and provenance.

preserve. Copy the source aggregate evaluation only when:

1. the source frame matches `bridge.from`,
2. all transport preconditions hold, and
3. `claim.semantic_requirements ∩ bridge.lose = ∅`.

Failure yields `N[transport_precondition]` or `N[transport_loss]`.

degrade. Attempt preservation. If relevant detail is lost, apply the default degradation:

```

T → N[transport_loss]
F → N[transport_loss]
B → B[boundary_mix]
N → N

```

Support becomes `partial` when truth degrades unless a truth policy overrides the default.

remap_recompute. Map the source claim through `claim_map`, complete the destination frame, then evaluate the mapped claim under the destination evaluator panel and resolution policy. This mode permits truth to change across the bridge.

7.2 Transport Result

```

TransportResult:
  claimId: string
  bridgeId: string
  status: metadata_only | pattern_only | preserved | degraded | transported | blocked
  dstPattern: FramePatternNode
  dstFrame: FrameNode?
  mappedClaim: ClaimNode | ExprNode?
  sourceAggregate: EvalNode

```

```

dstAggregate: EvalNode?
per_evaluator: { EvaluatorId: EvalNode }?
preserve: [string]
lose: [string]
gain: [string]
risk: [string]
provenance: [string]

```

`pattern_only` is a compatibility status indicating that a legacy or handler-less bridge exposed only its endpoint pattern. It is distinct from an explicitly declared `metadata_only` mode.

7.3 Transport Lint

Claims transported under `preserve` or `degrade` should declare `semantic_requirements`. An empty declaration triggers `lint.transport.semantic_requirements_empty` as a warning; the transport still runs.

8. Baselines, Unbound Behavior, and Emergence

A baseline is a named reference, not anonymous nothingness.

```

Baseline:
  id: BaselineId
  kind: point | set | manifold | moving
  criterion: Expr | BindingRef
  frame: Frame | FramePattern
  evaluation_mode: fixed | on_reference | tracked

```

Surface forms:

```

|0:nominal|
|0:metastable_B|
|0:rolling_reference|

```

If more than one baseline is active, bare `|0|` is illegal.

8.1 Session-Relative Baseline Timing

Baseline timing is defined relative to an evaluation session and effective step context.

```

effective_frame = merge(bundle.frame, session.base_frame, step.frame_override)
effective_time  = step.time ?? session.base_time ?? bundle.time ?? env.clock
effective_history = resolve(step.history_binding) if present, else env.history

```

A baseline-local frame overlays the effective step frame for that baseline's resolution only.

- **fixed, shared state** — cache key `(session_id, baseline_id)`. Resolve once for the session; later frame/time/history changes do not invalidate the cached value.
- **fixed, isolated state** — cache key `(session_id, step_id, baseline_id)`. Reinitialize each step.
- **on_reference** — resolve at each claim evaluation under the current step context. Memoization is permitted only for context-equivalent calls.
- **tracked** — resolve as a time-indexed reference; required for `kind=moving`.

A cyclic baseline dependency yields `N[undefined_term]`. A tracked trajectory used where a scalar is required must declare a reduction/comparison rule or the dependent claim yields `N[undefined_term]`.

8.2 Unbound Behavior

Unbound references are kinded:

```
|∞:asymptotic|
|∞:finite_time|
|∞:nonterminating|
|∞:externally_unbounded|
```

These distinguish asymptotic divergence, finite-time blow-up, nontermination, and absence of a bound in the active frame.

8.3 Emergence

```
Emergence:
  property: Expr
  onset: Condition
  persists_while: Condition?
  dissolves_when: Condition?
  hysteresis: Condition?
  witness: [ClaimId] = []
```

Emergence therefore represents onset, persistence, dissolution, and path dependence rather than only a crisp threshold.

9. Anchors, Adequacy, and Model-Status

An anchor is a modeling construct whose literal status and task-specific fitness are disclosed.

```
Anchor:
  id: AnchorId
  term: Term | ClaimId
  subtype: idealization | placeholder | proxy | aggregate
  status: active | inactive | counterfactual
  adequacy_policy: ResolutionPolicyRef?
  adequacy: [AdequacyAssessment]
  requires_joint_with: [AnchorId] = []
```

```
AdequacyAssessment:
  id: AdequacyAssessmentId
  task: Symbol
  producer: Symbol
  score: [0,1] | N?
  threshold: [0,1]
  method: BindingRef
  basis: [BindingRef | EvidenceRef | ClaimRef] = []
  confidence: [0,1]?
  failure_modes: [Symbol] = []
```

```
JointAdequacy:
  id: JointAdequacyId
  anchors: [AnchorId]
  adequacy_policy: ResolutionPolicyRef?
  assessments: [AdequacyAssessment]
```

The surface keyword `fictional_anchor` normalizes to `Anchor` (`subtype=idealization`) when subtype is omitted. An explicitly supplied subtype is preserved.

9.1 Assessment-Level Semantics

Each assessment is evaluated independently:

1. resolve method,
 2. resolve basis,
 3. determine or compute score,
 4. compare score to threshold.
- unresolved method or required basis $\rightarrow N[\text{missing_binding}]$
 - score=N $\rightarrow N[\text{not_yet_applicable}]$
 - numeric score below threshold $\rightarrow F[\text{threshold_not_met}]$
 - executable method materially disagrees with declared score $\rightarrow B[\text{method_conflict}]$

A numeric score never bypasses method resolvability.

9.2 Multiple Assessments Per Task

Multiple applicable assessments require an `adequacy_policy`. Without one, the aggregate adequacy result is `N[missing_policy]` and the evaluator emits `lint.adequacy.missing_policy_multi_assessment`.

9.3 Licensing Rules

The license task is selected in this order:

1. `claim.annotations["license_task"]`,
2. evaluator-policy mapping,
3. `ctx.frame.task`.

The active anchor set is the exact, deduplicated, order-insensitive set in `Claim.uses_anchors`.

Joint adequacy is required when a used anchor requires another used anchor or when a joint-adequacy record exists for the exact active set and task. Subsets and supersets do not match.

- required joint record missing $\rightarrow N[\text{missing_joint_adequacy}]$
- joint result below threshold $\rightarrow F[\text{joint_inadequacy}]$
- unresolved joint result $\rightarrow N[\text{not_yet_applicable}]$

9.4 License Result

```
LicenseResult:  
  claimId: string  
  licenseTask: Symbol  
  anchors: [AnchorId]  
  individual: { AnchorId: AdequacyResult }  
  joint: AdequacyResult?  
  overall: EvalNode
```

License aggregation uses operational severity, not propositional conjunction:

1. F if any required component is F,
2. else B if any is B,
3. else N if any is N,
4. else T.

A failed, conflicted, or unresolved license does not automatically change world-claim truth. It says the modeling fiction is not licensed under the active task and provenance conditions.

9.5 Circularity Rule

Adequacy basis references may cite claims, but cycles are prohibited. An assessment that directly or transitively depends on a claim which depends on that same assessment yields `N[circular_dependency]` and diagnostic `lint.adequacy.circular_basis`.

10. Judgments and Normative Terms

Normative predicates are criterion-bound through `judged_by`.

```
safe(grid_state) judged_by policy://grid/safety_margin_v3
```

`JudgedExpr` wraps any evaluable expression, not only a predicate. Evaluation is two-stage:

1. evaluate the inner expression under the active evaluator;
2. pass its truth, the expression, criterion reference, context, history, and services to the criterion binding.

`CriterionBindingContract`:

```
evalJudged(innerTruth, expr, criterionRef, ctx, history, services)
  → TruthCore
```

Missing or unresolved criterion binding yields `N[missing_binding]`. Per-evaluator judged results are aggregated normally.

11. Surface Syntax and Operator Kernel

11.1 Surface Sugar

```
[behavior] (subject)
local { ... }
systemic { ... }
meta { ... }
declared_as
within
emerges_when
fictional_anchor
```

11.2 Minimal Operator Kernel

Class	Unicode	Canonical ASCII
logical not	¬	NOT
logical and	∧	AND
logical or	∨	OR
implication	→	->
biconditional	↔	<=>
observational cause	⇒[obs]	=>[obs]
interventional cause	⇒[do]	=>[do]
approaches	→	-->
diverges	⇒[kind]	=>>[kind]
oscillates	↔	OSC
cycles	↻	CYC
transforms	↦	>
crosses	↔	><
hard boundary	⎔	
emergent	⚠	EMRG

Class	Unicode	Canonical ASCII
paradox mark	⊥	PARA
undefined mark	∅	UNDEF
null behavior	∅	NULL
refinement	⊆	<=
approximation	≈[metric,tol]	~=[metric,tol]

Each primitive has one canonical ASCII alias. $\Leftarrow$ is not in the kernel; use $\mapsto$ for transformation.

12. Worked Example

Bundle:

```
id: Grid_Contingency_01
frame:
  system: PowerGrid
  namespace: ACLoadFlow
  scale: micro
  task: operations
  regime: contingency
  version: v2
  facet_policy: fp_grid_ops
```

evaluators:

```
- id: ev_grid_model
  kind: model
  role: primary
  binding: model://grid/aclf/v2
  evidence_policy: policy://grid/evidence/default
  inference_policy: policy://grid/inference/default
  provenance_policy: policy://grid/provenance/default
```

resolution_policy:

```
kind: single
members: [ev_grid_model]
```

time:

```
kind: interval
start: 2026-03-06T09:00:00
end: 2026-03-06T09:00:30
```

baselines:

```
- id: margin
  kind: point
  criterion: ref b_margin_ref
  frame:
    system: PowerGrid
    namespace: ACLoadFlow
    scale: micro
    task: operations
    regime: contingency
  evaluation_mode: on_reference
```

```

evidence:
  - id: scada_bus7
    kind: measurement
    binding: data://scada/bus7
    observer: operator_A
    completeness: 0.93
    internal_conflict: 0.02

  - id: pmu_bus7
    kind: measurement
    binding: data://pmu/bus7
    observer: sensor_cluster_3
    completeness: 0.96
    internal_conflict: 0.01

evidence_relations:
  - id: er_bus7
    lhs: scada_bus7
    rhs: pmu_bus7
    kind: conflicts
    score: 0.72

anchors:
  - id: a_nminus1
    term: Nminus1
    subtype: idealization
    status: active
    adequacy:
      - id: aa_n1_pred
        task: prediction
        producer: ev_grid_model
        score: 0.98
        threshold: 0.95
        method: sim://checks/n1_pred
      - id: aa_n1_ctrl
        task: control
        producer: ev_grid_model
        score: 0.91
        threshold: 0.90
        method: sim://checks/n1_ctrl
      - id: aa_n1_expl
        task: explanation
        producer: ev_grid_model
        score: 0.63
        threshold: 0.75
        method: audit://postmortem/n1_expl

bridges:
  - id: b_micro_to_regional
    from: @{system=PowerGrid, namespace=ACLoadFlow, scale=micro, task=operations,
    to: @{system=PowerGrid, namespace=PlanningModel, scale=regional, task=planning
    via: model://aggregate_flow_map
    preserve: [power_balance]
    lose: [phase_angle, switching_order]

```

```

risk: [aggregation_reversal]
transport:
  mode: metadata_only

```

Authored claims:

```

local {
  c1: overload(line_B);
  c2: overload(line_B) =>[obs] voltage_drop(bus_7)
    refs [scada_bus7, pmu_bus7];
}

systemic {
  c3: voltage_instability EMRG
    when reactive_margin --> |0:margin|
    while demand_ramp_gt(0.02_pu_per_min)
    until load_shed(zone_2)
    uses [a_nminus1];
}

meta {
  c4: declare Nminus1 as idealization;
  c5: note "N-1 is acceptable for dispatch prediction but weak as a restoration expl
}

```

Illustrative results:

```

eval(c1) = T / supported
eval(c2) = B[source_conflict] / conflicted
eval(c3) = T / partial
adequacy(a_nminus1, prediction) = T
adequacy(a_nminus1, control) = T
adequacy(a_nminus1, explanation) = F[threshold_not_met]
block(local) = T ^ B = B
block(systemic) = T

```

13. Lint Rules

1. Every bundle declares at least one typed evaluator and a resolution policy.
2. Every evaluable claim has an explicit or inherited frame.
3. Every frame pattern used for direct evaluation resolves to a full frame.
4. Legacy @System:Namespace::Scope warns unless completed.
5. Every required external reference resolves at evaluation time.
6. B and N always carry reason codes.
7. Block truth uses Section 5 conjunction, folded per evaluator before aggregation.
8. A block with no evaluable claims resolves to N[empty_block].
9. Bare |0| is illegal when more than one baseline is active.
10. kind=moving baselines use evaluation_mode=tracked.
11. Bare |∞| is illegal unless kind is declared.
12. =>[do] requires an intervention target or binding.
13. Every active anchor declares an adequacy assessment for the current task.
14. Claims materially dependent on anchors should declare uses_anchors.
15. Required joint adequacy is checked by exact active-anchor set and task.
16. Missing required joint adequacy yields N[missing_joint_adequacy].
17. Failed required joint adequacy yields F[joint_inadequacy] for the composition judgment.

18. Every bridge declares `preserve` and `lose`.
19. Projection may describe a coarser pattern, but truth transport without declared independence or a bridge is illegal.
20. Normative predicates require `judged_by`.
21. Free prose in meta uses `note(...)`.
22. Internal evidence conflict belongs on `Evidence`; cross-evidence conflict belongs in `EvidenceRelation` or `provenance`.
23. Preserve/degrade transport with empty `semantic_requirements` emits a warning.
24. Multiple same-task adequacy assessments without `adequacy_policy` emit a warning and resolve `N[missing_policy]`.
25. Circular adequacy basis emits an error and resolves `N[circular_dependency]`.

13.1 Normative Diagnostic Codes

- **Rule 23 — `lint.transport.semantic_requirements_empty`**
Severity: warning; phase: transport. The warning does not stop transport.
- **Rule 24 — `lint.adequacy.missing_policy_multi_assessment`**
Severity: warning; phase: license. The affected adequacy aggregate becomes `N[missing_policy]`.
- **Rule 25 — `lint.adequacy.circular_basis`**
Severity: error; phase: license. The affected assessment becomes `N[circular_dependency]`.

Unless otherwise stated, lint warnings do not alter truth by themselves. Lint errors may localize evaluation to `N[...]` when the underlying semantic rule requires failure.

14. Open Extensions

The following remained extensions beyond the v0.2.2 core:

- quantifiers and aggregation (`forall`, `exists`, proportions, cohorts),
- probabilistic semantics beyond support and confidence,
- full deontic logic for obligation, permission, and prohibition,
- richer domain-specific facet policies and dependency algebras,
- proof obligations for chained transports,
- richer model-license propagation into downstream tooling,
- user-defined summaries distinct from normative block folding,
- and richer evidence inference beyond declared relations.

15. How to Read Limnalis

Limnalis is a disclosure-oriented specification layer for complex systems. It is especially useful where claims depend on frame, assumptions, evaluator, evidence, and abstraction boundary. The net effect is simple: it becomes much harder to smuggle universality, idealization, or ambiguity into a statement without declaring it.

Appendix A: Grammar and Canonical AST

A.1 Parsing Model

Limnalis uses three stages:

1. **Parse** — source text to a raw tree preserving spelling, shorthand, block order, and source spans.
2. **Normalize** — aliases and shorthand to canonical AST.
3. **Resolve / validate** — resolve references, complete evaluation frames, apply defaults, and emit diagnostics.

A.2 Lexical Profile

```
Ident      ::= Letter { Letter | Digit | "_" | "-" } ;
Number     ::= ["-"] Digit { Digit } [ "." Digit { Digit } ] ;
String     ::= '"' { Char | Escape } '"' ;
Boolean    ::= "true" | "false" ;
Uri        ::= Scheme "://" UriChar { UriChar } ;
Symbol     ::= Ident | String ;
Ref        ::= Ident | Uri | String ;
ListLiteral ::= "[" [ Value { "," Value } ] "]" ;
MapLiteral ::= "{" [ MapEntry { "," MapEntry } ] "}" ;
MapEntry   ::= (Ident | String) ":" Value ;
Value      ::= Number | Boolean | String | Uri | Symbol
              | ListLiteral | MapLiteral | FramePattern ;
Comment    ::= "#" { not_newline } | "/" { not_newline } ;
```

A.3 Top-Level Structure

```
Document    ::= BundleDecl EOF ;
BundleDecl  ::= "bundle" Ident "{" BundleItem* "}" ;
BundleItem  ::= FrameDecl
              | EvaluatorDecl
              | ResolutionPolicyDecl
              | TimeDecl
              | BindingDecl
              | FacetPolicyDecl
              | AssumptionDecl
              | BaselineDecl
              | EvidenceDecl
              | EvidenceRelationDecl
              | AnchorDecl
              | JointAdequacyDecl
              | BridgeDecl
              | ClaimBlock ;
```

A.4 Frames

```
FrameDecl   ::= "frame" ( FrameBlock | FramePattern ) ";"? ;
FrameBlock  ::= "{" FrameField* "}" ;
FrameField  ::= "system" Symbol ";"
              | "namespace" Symbol ";"
              | "scale" Symbol ";"
              | "task" Symbol ";"
              | "regime" Symbol ";"
              | "observer" Symbol ";"
              | "version" Symbol ";"
              | "facet_policy" Ref ";" ;
FramePattern ::= "@{" [ FacetAssign { "," FacetAssign } ] "}"
              | "@" Symbol ":" Symbol "::" Symbol ;
FacetAssign ::= FacetName "=" Symbol ;
FacetName   ::= "system" | "namespace" | "scale" | "task"
              | "regime" | "observer" | "version" ;
```

A.5 Evaluators, Policies, Time, Bindings, and Facet Policies

```
EvaluatorDecl      ::= "evaluator" Ident "{" EvaluatorField* "}" ;
EvaluatorField     ::= "kind" EvaluatorKind ";"
                  | "role" EvaluatorRole ";"
                  | "binding" Ref ";"
                  | "evidence_policy" Ref ";"
                  | "inference_policy" Ref ";"
                  | "provenance_policy" Ref ";" ;
EvaluatorKind      ::= "model" | "human" | "agent" | "institution" | "ensemble" | "proc
EvaluatorRole      ::= "primary" | "adversarial" | "audit" | "auxiliary" ;

ResolutionPolicyDecl ::= "resolution_policy" Ident "{"
                        "kind" ResolutionKind ";"
                        [ "members" RefList ";" ]
                        [ "order" RefList ";" ]
                        [ "binding" Ref ";" ]
                        "}" ;
ResolutionKind     ::= "single" | "paraconsistent_union" | "priority_order" | "adjudicat

TimeDecl           ::= "time" TimeSpec ";" | "time" "{" TimeField* "}" ;
TimeSpec           ::= "point" "(" Timestamp ")"
                  | "interval" "[" Timestamp "," Timestamp "]"
                  | "window" "[" Timestamp "," Timestamp "]"
                  [ "lag" Duration ] [ "step" Duration ] ;
TimeField          ::= "kind" ( "point" | "interval" | "window" ) ";"
                  | "t" Timestamp ";" | "start" Timestamp ";" | "end" Timestamp ";"
                  | "lag" Duration ";" | "step" Duration ";" ;

BindingDecl        ::= "binding" Ident "{" BindingField* "}" ;
BindingField       ::= "kind" BindingKind ";"
                  | "target" Ref ";" | "version" Symbol ";" | "hash" String ";" ;
BindingKind        ::= "equation" | "dataset" | "code" | "model"
                  | "document" | "policy" | "ontology" ;

FacetPolicyDecl    ::= "facet_policy" Ident "{" FacetPolicyField* "}" ;
FacetPolicyField   ::= "order" "{" FacetOrderField* "}"
                  | "independent" PairList ";"
                  | "depends_on" PairList ";" ;
FacetOrderField    ::= FacetName ( "eq" | Ref ) ";" ;
PairList           ::= "[" [ FacetPair { "," FacetPair } ] "]" ;
FacetPair          ::= "(" FacetName "," FacetName ")" ;
```

A.6 Assumptions, Baselines, and Evidence

```
AssumptionDecl     ::= "assumption" Ident "{"
                    "expr" Expr ";"
                    "status" AssumptionStatus ";"
                    [ "refs" RefList ";" ]
                    "}" ;
AssumptionStatus   ::= "active" | "suspended" | "counterfactual" ;

BaselineDecl       ::= "baseline" Ident "{"
                    "kind" BaselineKind ";"
```

```

        "criterion" CriterionSpec ";"
        "frame" ( FrameBlock | FramePattern ) ";"?
        [ "evaluation_mode" BaselineMode ";" ]
    }" ;
BaselineKind      ::= "point" | "set" | "manifold" | "moving" ;
BaselineMode      ::= "fixed" | "on_reference" | "tracked" ;
CriterionSpec     ::= "expr" Expr | "ref" Ref ;

EvidenceDecl      ::= "evidence" Ident "{"
    "kind" EvidenceKind ";"
    "binding" Ref ";"
    [ "observer" Symbol ";" ]
    [ "time" TimeSpec ";" ]
    [ "completeness" Number ";" ]
    [ "internal_conflict" Number ";" ]
    }" ;
EvidenceKind      ::= "measurement" | "dataset" | "testimony"
    | "simulation" | "audit" | "derived" ;

EvidenceRelationDecl ::= "evidence_relation" Ident "{"
    "lhs" Ref ";"
    "rhs" Ref ";"
    "kind" EvidenceRelationKind ";"
    [ "score" Number ";" ]
    [ "refs" RefList ";" ]
    }" ;
EvidenceRelationKind ::= "corroborates" | "conflicts" | "depends_on" | "duplicate_of"

```

A.7 Anchors, Adequacy, and Bridges

```

AnchorDecl        ::= ( "anchor" | "fictional_anchor" ) Ident "{"
    "term" TermSpec ";"
    [ "subtype" AnchorSubtype ";" ]
    [ "status" AnchorStatus ";" ]
    [ "requires_joint_with" RefList ";" ]
    [ "adequacy_policy" Ref ";" ]
    AdequacyAssessmentDecl*
    }" ;
AnchorSubtype     ::= "idealization" | "placeholder" | "proxy" | "aggregate" ;
AnchorStatus      ::= "active" | "inactive" | "counterfactual" ;
TermSpec          ::= "symbol" Symbol | "claim" Ref | "expr" Expr ;

AdequacyAssessmentDecl ::= "assessment" Ident "{"
    "task" Symbol ";"
    "producer" Symbol ";"
    [ "score" ( Number | "N" ) ";" ]
    "threshold" Number ";"
    "method" Ref ";"
    [ "basis" RefList ";" ]
    [ "confidence" Number ";" ]
    [ "failure_modes" RefList ";" ]
    }" ;

JointAdequacyDecl ::= "joint_adequacy" Ident "{"

```

```

        "anchors" RefList ";"
        [ "adequacy_policy" Ref ";" ]
        AdequacyAssessmentDecl*
    }" ;

BridgeDecl ::= "bridge" Ident "{"
    "from" FramePattern ";"
    "to" FramePattern ";"
    "via" Ref ";"
    "preserve" RefList ";"
    "lose" RefList ";"
    [ "gain" RefList ";" ]
    [ "risk" RefList ";" ]
    [ TransportDecl ]
    }" ;

TransportDecl ::= "transport" "{"
    "mode" TransportMode ";"
    [ "claim_map" Ref ";" ]
    [ "truth_policy" Ref ";" ]
    [ "preconditions" RefList ";" ]
    [ "dst_evaluators" RefList ";" ]
    [ "dst_resolution_policy" Ref ";" ]
    }" ;

TransportMode ::= "metadata_only" | "preserve" | "degrade" | "remap_recompute" ;

```

A.8 Claim Blocks and Claims

```

ClaimBlock ::= Stratum [ Ident ] "{" ClaimDecl* }" ;
Stratum    ::= "local" | "systemic" | "meta" ;
ClaimDecl  ::= ClaimShort | ClaimLong ;
ClaimShort ::= Ident ":" Expr ClaimTail* ";" ;
ClaimTail  ::= "uses" RefList
    | "refs" RefList
    | "requires" RefList
    | "annotations" MapLiteral ;
ClaimLong  ::= "claim" Ident "{"
    "expr" Expr ";"
    [ "uses_anchors" RefList ";" ]
    [ "semantic_requirements" RefList ";" ]
    [ "refs" RefList ";" ]
    [ "annotations" MapLiteral ";" ]
    }" ;
RefList    ::= "[" [ Ref { "," Ref } ] "]" ;

```

A.9 Expression Grammar

```

Expr      ::= JudgedExpr ;
JudgedExpr ::= LogicalExpr [ "judged_by" Ref ] ;
LogicalExpr ::= IffExpr ;
IffExpr    ::= ImplExpr { IffOp ImplExpr } ;
ImplExpr   ::= OrExpr { ImplOp OrExpr } ;
OrExpr     ::= AndExpr { OrOp AndExpr } ;
AndExpr    ::= UnaryExpr { AndOp UnaryExpr } ;
UnaryExpr  ::= [ NotOp ] CoreExpr ;

```



```

NotOp          ::= "¬" | "NOT" ;
AndOp          ::= "∧" | "AND" ;
OrOp           ::= "∨" | "OR" ;
ImplOp         ::= "→" | "→" ;
IffOp          ::= "↔" | "<=>" ;

CoreExpr       ::= CausalExpr | EmergenceExpr | DeclarationExpr
                  | NoteExpr | DynamicExpr | PredicateExpr | "(" Expr ")" ;

CausalExpr     ::= SimpleExpr CausalOp SimpleExpr [ InterventionClause ] ;
CausalOp       ::= "⇒[obs]" | "⇒[obs]" | "⇒[do]" | "⇒[do]" ;
InterventionClause ::= "intervention" ( Ref | "(" Expr ")" ) ;

EmergenceExpr  ::= PropertyExpr EmergenceOp "when" Expr
                  [ "while" Expr ] [ "until" Expr ]
                  [ "hysteresis" Expr ] [ "witness" RefList ] ;
EmergenceOp    ::= "Δ" | "EMRG" ;

DeclarationExpr ::= "declare" Term "as" Symbol
                  [ "within" ( FramePattern | "(" Expr ")" ) ]
                  | Term DeclaredAsOp Symbol
                  [ "within" ( FramePattern | "(" Expr ")" ) ] ;
DeclaredAsOp   ::= "declared_as" | "declared-as" ;
NoteExpr       ::= "note" String ;

DynamicExpr    ::= Term DynamicOp [ TermOrExpr ] ;
DynamicOp      ::= "→" | "-->"
                  | ( "⇒" | "⇒>>" ) "[" Symbol "]"
                  | "↔" | "OSC" | "J" | "CYC"
                  | "↗" | "|>" | "↘" | "><" ;

PredicateExpr  ::= Symbol [ "(" [ ArgList ] ")" ] ;
SimpleExpr     ::= DynamicExpr | PredicateExpr | "(" Expr ")" ;
PropertyExpr   ::= PredicateExpr | "(" Expr ")" ;
ArgList        ::= Arg { "," Arg } ;
Arg            ::= Term | "(" Expr ")" ;
TermOrExpr     ::= Term | "(" Expr ")" ;
Term           ::= BaselineRef | UnboundRef | NullTerm | Number | String
                  | Boolean | Uri | Symbol | ListLiteral | "(" Term ")" ;
BaselineRef    ::= "|0:" Ident "|" ;
UnboundRef     ::= "|∞:" Ident "|" | "|inf:" Ident "|" ;
NullTerm       ::= "Q" | "NULL" ;

```

A.10 Canonical AST Nodes

All canonical AST classes carry a discriminator field node and may carry source span information.

```

BundleNode:
  node: Bundle
  id: string
  frame: FrameNode | FramePatternNode
  evaluators: [EvaluatorNode]
  resolutionPolicy: ResolutionPolicyNode
  time: TimeCtxNode?

```

```

bindings: [BindingNode]
facetPolicies: [FrameFacetPolicyNode]
assumptions: [AssumptionNode]
baselines: [BaselineNode]
evidence: [EvidenceNode]
evidenceRelations: [EvidenceRelationNode]
anchors: [AnchorNode]
jointAdequacies: [JointAdequacyNode]
bridges: [BridgeNode]
claimBlocks: [ClaimBlockNode]

```

Key settled nodes:

ResolutionPolicyNode:

```

node: ResolutionPolicy
id: string
kind: single | paraconsistent_union | priority_order | adjudicated
members: [string]?
order: [string]?
binding: string?

```

TransportNode:

```

node: Transport
mode: metadata_only | preserve | degrade | remap_recompute
claimMap: string?
truthPolicy: string?
preconditions: [string]
dstEvaluators: [string]
dstResolutionPolicy: string?

```

AdequacyAssessmentNode:

```

node: AdequacyAssessment
id: string
task: string
producer: string
score: number | N?
threshold: number
method: string
basis: [string]
confidence: number?
failureModes: [string]

```

ClaimBlockNode:

```

node: ClaimBlock
id: string
stratum: local | systemic | meta
claims: [ClaimNode]

```

ClaimNode:

```

node: Claim
id: string
kind: atomic | causal | dynamic | emergence | declaration | judgment | note | logi
expr: ExprNode
usesAnchors: [string]
semanticRequirements: [string]

```

```
refs: [string]
annotations: Map<string, Value>
```

Expression nodes:

```
PredicateExprNode:
  node: PredicateExpr
  name: string
  args: [TermNode]
```

```
LogicalExprNode:
  node: LogicalExpr
  op: not | and | or | implies | iff
  args: [ExprNode]
```

```
CausalExprNode:
  node: CausalExpr
  mode: obs | do
  lhs: ExprNode
  rhs: ExprNode
  intervention: ExprNode | string?
```

```
DynamicExprNode:
  node: DynamicExpr
  op: approaches | diverges | oscillates | cycles | transforms | crosses
  subject: TermNode
  target: TermNode | ExprNode?
  qualifiers: Map<string, Value>?
```

```
EmergenceExprNode:
  node: EmergenceExpr
  property: ExprNode
  onset: ExprNode
  persistsWhile: ExprNode?
  dissolvesWhen: ExprNode?
  hysteresis: ExprNode?
  witness: [string]
```

```
DeclarationExprNode:
  node: DeclarationExpr
  term: TermNode
  declaredAs: string
  within: FramePatternNode | ExprNode?
```

```
JudgedExprNode:
  node: JudgedExpr
  expr: ExprNode
  criterionRef: string
```

```
NoteExprNode:
  node: NoteExpr
  text: string
```

`DynamicExprNode.op` is closed, not an arbitrary string. `|>/ $\mapsto$` normalizes to `transforms`.

A.11 Normalization Rules

- Unicode and ASCII aliases normalize to canonical operator names.
- `@System:Namespace::Scope` normalizes to a partial `FramePatternNode`.
- unnamed blocks receive deterministic synthetic IDs such as `local#1`.
- bare symbols in expression position become zero-arity predicates; in term position they remain symbol terms.
- `judged_by` normalizes to `JudgedExprNode`.
- `fictional_anchor` normalizes to `AnchorNode`; omitted subtype defaults to idealization.
- a legacy single evaluator normalizes to a one-member evaluator panel and single policy.
- bridge without transport normalizes to `metadata_only`.
- `|0:id|` normalizes to `BaselineRefTermNode`.
- `|∞:kind| / |inf:kind|` normalizes to `UnboundRefTermNode`.
- evaluable claim kind is inferred from expression node.

A.12 Resolution Rules

- direct claim evaluation requires a resolved full frame;
- bridge endpoints may remain partial patterns;
- omitted bridge facets are unspecified rather than invalid;
- destination patterns complete only when transport is applied;
- all required local and external references must resolve before dependent semantics are executed.

16. Reference Evaluator

16.1 Purpose

The reference evaluator is the normative orchestration layer for v0.2.2. It is normative about evaluation order, reference resolution, frame handling, baseline timing, adequacy and licensing, evidence-view materialization, per-evaluator claim evaluation, resolution policy aggregation, block folding, diagnostics, and step-scoped transport.

Domain truth computation may be delegated to bindings. A conformant implementation may optimize internally but must preserve observable results and diagnostics for the same canonical AST, environment, and bindings.

16.2 Inputs and Outputs

```
EvaluationRequest:  
  bundle: BundleNode  
  env: EvaluationEnvironment  
  sessions: [EvaluationSession] = [implicit_default_session]
```

```
EvaluationEnvironment:  
  bindingResolver: Resolver  
  frameResolver: Resolver?  
  history: OpaqueHistory  
  clock: TimeCtxNode?  
  cache: OpaqueCache?  
  policyOverrides: [string]?
```

```
EvaluationSession:  
  id: SessionId  
  shared_state: true | false = true  
  base_frame: FrameNode | FramePatternNode?  
  base_time: TimeCtxNode?  
  steps: [EvaluationStep]
```

```

EvaluationStep:
  id: StepId
  time: TimeCtxNode?
  history_binding: BindingRef?
  frame_override: FrameNode | FramePatternNode?
  claim_subset: [ClaimId]?
  transport_queries: [TransportQuery] = []

```

A one-shot evaluation is one implicit session with one implicit step.

```

EvaluationResult:
  bundleId: string
  sessions: [SessionResult]
  diagnostics: [Diagnostic]

```

```

SessionResult:
  id: SessionId
  baselineStore: [BaselineState]
  adequacyCache: [AdequacyResult]
  steps: [StepResult]
  diagnostics: [Diagnostic]

```

```

StepResult:
  id: StepId
  context: ContextSnapshot
  claims: [ClaimResult]
  blocks: [BlockResult]
  transports: [TransportResult]
  diagnostics: [Diagnostic]

```

```

Diagnostic:
  id: string
  severity: info | warning | error
  phase: resolve | frame | baseline | license | evidence | claim | block | transport
  subject: string
  code: string
  message: string

```

16.3 Effective Step Context

```

effective_frame = merge(bundle.frame, session.base_frame, step.frame_override)
effective_time   = step.time ?? session.base_time ?? bundle.time ?? env.clock
effective_history = resolve(step.history_binding) if present, else env.history

```

Later frame assignments override earlier ones. Direct evaluation requires a full resolved frame. `claim_subset` limits evaluated claims but does not itself force eager baseline resolution.

16.4 Runtime Artifacts

```

ResolutionStore:
  bindings: { ref: ResolvedArtifact | ResolutionFailure }
  policies: { id: ResolvedPolicy }
  evidence: { id: EvidenceNode }
  anchors: { id: AnchorNode }
  baselines: { id: BaselineNode }
  bridges: { id: BridgeNode }

```

```

    facetPolicies: { id: FrameFacetPolicyNode }

BaselineState:
  baselineId: string
  mode: fixed | on_reference | tracked
  status: ready | deferred | unresolved
  value: any?
  provenance: [string]

ClaimEvidenceView:
  claimId: string
  explicitEvidence: [EvidenceNode]
  relatedEvidence: [EvidenceNode]
  relations: [EvidenceRelationNode]
  crossConflictScore: number?
  completenessSummary: number?

AdequacyResult:
  source: string
  truth: T | F | B | N
  reason: string?
  provenance: [string]

TruthCore:
  truth: T | F | B | N
  reason: string?
  provenance: [string]

SupportResult:
  support: supported | partial | conflicted | absent | inapplicable
  confidence: number?
  provenance: [string]

```

16.5 Bound Handler Contracts

```

EvaluatorBindingContract:
  evalPredicate(expr, ctx, history) → TruthCore
  evalDynamic(expr, ctx, history, services) → TruthCore
  evalCausal(expr, ctx, history, services) → TruthCore
  evalEmergence(expr, ctx, history, services) → TruthCore
  evalDeclaration(expr, ctx, history, services) → TruthCore
  resolveBaselineCriterion(spec, ctx, history, services) → BaselineResolution
  assessSupport(claim, evidenceView, truthCore, ctx) → SupportResult
  assessConfidence(claim, evidenceView, truthCore, ctx) → number?

CriterionBindingContract:
  evalJudged(
    innerTruth, expr, criterionRef,
    ctx, history, services
  ) → TruthCore

BridgeBindingContract:
  transportClaim(
    claimResult, bridge, dstPattern,

```

```
    ctx, history, services
) → TransportResult
```

```
AdjudicatedResolutionContract:
    resolve(per_evaluator_evals, ctx) → EvalNode
```

Missing required handlers localize the affected evaluation to `N[missing_binding]`.

16.6 Normative Phase Order

For each session and then each step:

1. resolve references and policies;
2. build the effective step context;
3. initialize or reuse the baseline service;
4. evaluate relevant adequacy sets and compose claim-local licenses;
5. materialize declared evidence views per claim;
6. classify claims and evaluate each evaluable expression per evaluator;
7. synthesize support per evaluator;
8. assemble per-evaluator evaluations and apply the resolution policy;
9. fold blocks per evaluator, then aggregate block results;
10. execute step-scoped transport queries.

Adequacy is evaluated before dependent claims. Transport runs only after source claim results exist.

16.7 Phase Details

Phase 1 — Reference Resolution

Resolve local IDs, binding references, policy references, baseline criteria, adequacy methods and basis, criterion refs, and transport refs. Record structured failures in the resolution store. Failures remain localized.

Phase 2 — Step Context

Merge bundle/session/step frame, time, and history; activate evaluator panel and resolution policy; resolve direct-evaluation frame to full `FrameNode`.

Phase 3 — Baseline Service

Apply the session-relative fixed/on-reference/tracked rules from Section 8. A baseline may resolve eagerly or at first use as long as observable behavior is equivalent.

Phase 4 — Adequacy and Licensing

Evaluate each assessment independently, aggregate same-task assessments under `adequacy_policy`, detect method conflict and circularity, select license task, perform exact-set joint lookup, then compose `LicenseResult` using operational severity. Licensing does not overwrite world truth.

Phase 5 — Evidence View

Resolve claim evidence refs, collect declared relations, compute maximum relevant conflict score and minimum declared completeness. This is a per-claim view. Evaluator-specific support policy enters later.

Phase 6 — Claim Truth Per Evaluator

`NoteExpr` is non-evaluable and receives no per-evaluator or aggregate evaluation. Other expressions dispatch as follows:

- `PredicateExpr`, `DynamicExpr`, `CausalExpr`, and `EmergenceExpr` delegate to evaluator bindings;
- `DeclarationExpr` may delegate, otherwise defaults to T when no `within`, frame-match T/F for a pattern, or inherits an inner expression's truth;
- `JudgedExpr` evaluates its inner expression and then invokes the criterion binding;
- `LogicalExpr` is internal and uses Section 5 truth functions.

When a logical composition yields B or N, inherit a uniquely determining child reason when possible; otherwise use `logical_composition` and record contributing reasons in diagnostics.

Phase 7 — Support Per Evaluator

Default support policy:

- declaration with no evidence → `inapplicable`;
- no evidence → `absent`;
- declared relevant conflict → `conflicted`;
- incomplete or internally conflicted evidence without cross-conflict → `partial`;
- otherwise → `supported`.

Non-evaluable notes do not enter support synthesis. Evidence policy bindings may override the default. Confidence is unset unless assigned by policy or evaluator.

Phase 8 — Aggregate Claims

Apply the bundle resolution policy to the complete per-evaluator `EvalNode` map. Built-ins aggregate truth, reason, support, confidence, and provenance according to Section 4.3.

Phase 9 — Fold Blocks

For each evaluator, fold evaluator-local claim truths using Section 5 conjunction. Then aggregate the evaluator-local block results under the resolution policy. For adjudicated block aggregation, pass synthetic `EvalNodes` with evaluator-local block truth, `support=inapplicable`, and provenance including evaluator and block IDs.

Phase 10 — Transport

Locate source claim and bridge, match source frame, select transport mode, and execute the rules in Section 7. A bridge queried for truth without a required handler yields `N[transport_missing]`.

16.8 Failure Modes

Hard validation failures may abort evaluation:

- malformed canonical AST,
- missing bundle frame or evaluator panel,
- duplicate IDs in the same bundle-local namespace.

Localized failures include missing bindings, unresolved baseline criteria, missing licensing data, unsupported criteria, unsafe projection, missing transport handler, and circular adequacy basis. They produce reason-bearing N results without poisoning unrelated claims.

16.9 Conformance Rules

A v0.2.2 evaluator:

1. consumes canonical AST or an equivalent normalized representation;
2. preserves normative phase order;
3. implements the four-valued truth functions;

4. keeps bridge endpoints as patterns until full destination evaluation is needed;
5. resolves baseline timing relative to sessions;
6. keeps fiction licensing distinct from world truth;
7. materializes evidence conflict before support;
8. requires reasons for B and N;
9. never silently turns missing information into F;
10. evaluates claims per evaluator before aggregation;
11. folds blocks per evaluator before block aggregation;
12. applies declared transport modes;
13. evaluates adequacy before dependent claims;
14. enforces exact-set joint matching and adequacy circularity rules.

17. Conformance Corpus

The corpus is a set of machine-checkable spec fixtures rather than tutorial examples. Deterministic `test : // bindings` pin down semantics.

17.1 Fixture Conventions

Representative fixtures include:

- `test://eval/declaration_v1` — default declaration behavior;
- `test://eval/atoms_v1` — `p=T, q=T, b=B[source_conflict], n=N[undefined_term]`;
- `test://eval/baseline_v1` — deterministic baseline comparison;
- `test://eval/grid_v1` — grid truth fixture;
- `test://eval/jwt_gateway_v1` — JWT truth fixture;
- `test://policy/auth_access_v3` — judged access criterion;
- `test://policy/jwt_support_v1` — reference-default support behavior;
- `test://eval/adversarial_v1` — returns F for fixture predicates;
- `test://resolution/adjudicated_v1` — deterministic adjudication;
- `test://bridge/pattern_only, pass_through, degrade_v1, remap_v1`;
- `test://baseline/const10, series_9_10_11, reactive_margin_zero`;
- `test://adequacy/recompute_v1` — recomputes 0.88 and can expose method conflict.

17.2 Track A — Core Semantics

- **A1 Resolved shorthand frame.** Pattern completes to a full frame; declaration matching yields T and nonmatching task yields F.
- **A2 Unresolved shorthand frame.** Direct evaluation aborts at frame resolution; no claim/block evals.
- **A3 Logical composition and block folding.** Pins $B \wedge N = F$, per-block folding, and `logical_composition` diagnostics.
- **A4 Baseline modes.** Fixed, on-reference, tracked, and invalid moving/fixed combination.
- **A5 Evidence conflict vs partial support.** Truth remains T while support is `conflicted` or `partial`.
- **A6 Individual and joint adequacy.** Exact-set joint matching; missing joint record yields `N[missing_joint_adequacy]`.
- **A7 Pattern-only vs transported bridge.** Distinguishes metadata/pattern plumbing from actual truth transport.
- **A8 Multi-evaluator conflict.** T plus F under paraconsistent union yields `B[evaluator_conflict]`; blocks fold per evaluator first.
- **A9 Priority-order resolution.** First non-N evaluator wins.
- **A10 Transport truth modes.** Preserve, degrade, and remap/recompute; includes empty semantic-requirements warning path.
- **A11 Session-based baseline timing.** Shared-state fixed baseline retains step-1 value while on-reference baseline re-resolves; isolated-state fixed baseline reinitializes per step.
- **A12 Adequacy method conflict and circularity.** Declared/computed disagreement yields `B[method_conflict]`; multiple assessments without policy yield `N[missing_policy]`; circular basis yields `N[circular_dependency]`.

- **A13 Core JudgedExpr.** Inner expression is evaluated before criterion binding; missing criterion yields `N[missing_binding]`.
- **A14 Adjudicated resolution.** A binding consumes complete per-evaluator eval maps at both claim and block levels.

17.3 Track B — Domain Bundles

- **B1 Grid contingency.** Exercises causal claims, emergence, baseline resolution, evidence conflict, task-specific anchor adequacy, and transport metadata.
- **B2 JWT access and adequacy.** Exercises criterion-bound access judgment, evidence-policy override, stateless-session and clock-skew idealizations, and the separation of world truth from revocation-task licensing.

The complete canonical source, normalized-AST expectations, evaluation expectations, and diagnostic expectations are preserved in the companion conformance matrix and machine-readable fixture corpus.

18. Schema Package

The v0.2.2 schema package is derived from the conformance corpus rather than prose alone.

- `limnalis_ast_schema_v0.2.2.json` — canonical normalized AST;
- `limnalis_conformance_result_schema_v0.2.2.json` — evaluator/conformance output;
- `limnalis_fixture_corpus_schema_v0.2.2.json` — machine-readable corpus;
- `limnalis_schema_validation_report_v0.2.2.json` — validation result.

Settled schema decisions:

1. `ResolutionPolicyNode` is a discriminated union with mode-specific required and forbidden fields.
2. `TransportNode` is a discriminated union.
3. `AdequacyAssessmentNode.score` is optional because executable methods may compute it.
4. `Evaluable ClaimResult` and `BlockResult` expose explicit `per_evaluator` maps.
5. `NoteExpr` results omit per-evaluator and aggregate evaluations.

Semantic constraints intentionally left to evaluator/custom validation include exact-set joint matching, cross-namespace ID uniqueness, equality of priority-order member/order sets, and cross-node cycle detection.

End of reconstructed Limnalis v0.2.2 specification.